

笔记

11.14

一. 数据库的语言标准

SQL: 结构化查询语言

DCL: 数据库控制语句 【授权 grant 收回权限 revoke】

DDL: 数据库定义语句 【操作数据库对象（表、索引、视图、存储过程）】

Create alter drop

DML: 数据库操作语句 【操作数据（一行数据记录）】

Insert update delete

二、修改表

```
ALTER TABLE employee ADD bm_id INT;
ALTER TABLE employee ADD bm_id INT FIRST;
ALTER TABLE employee ADD bm_id INT AFTER name;
/* 删除字段 */
ALTER TABLE employee DROP bm_id
/* 删除表 */
DROP TABLE employee;
/* 删除数据库 */
DROP database world;
```

三、主外键依赖

1. 创建主表

```
////////////////////////////////////
CREATE TABLE `dept` (
  `dp_id` int(11) NOT NULL,
  `dp_name` varchar(128) NOT NULL,
  PRIMARY KEY (`dp_id`),
  UNIQUE KEY `dp_name` (`dp_name`)
) ;
////////////////////////////////////
CREATE TABLE dept
(
dp_id INT PRIMARY KEY NOT NULL,
dp_name VARCHAR(128) NOT NULL UNIQUE
);
```

3. 创建外键

```
ALTER TABLE employee ADD CONSTRAINT fk_dp_emp FOREIGN KEY (bm_id)
REFERENCES dept(dp_id);
```

```
CREATE TABLE `employee` (
  `id` int(11) NOT NULL AUTO_INCREMENT,
  `name` varchar(30) DEFAULT 'NoSet',
  `id_number` char(18) NOT NULL,
  `rzsj` datetime DEFAULT NULL,
  `gender` char(8) DEFAULT '男',
  `salary` decimal(8,2) DEFAULT NULL,
  `bm_id` int(11) DEFAULT NULL,
  PRIMARY KEY (`id`),
  CONSTRAINT `fk_dp_emp` FOREIGN KEY (`bm_id`) REFERENCES `dept`
  (`dp_id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8;
```

4. 验证主外键约束

5. 其他约束

Unique 唯一键

四、排序

ORDER BY 子句

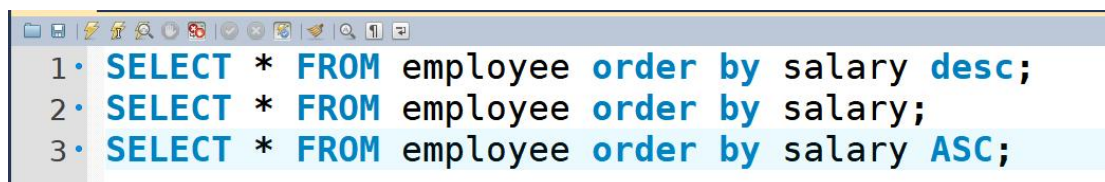
主要用于对数据进行排序。

格式: order by 字段 1 [asc|desc], 字段 2 [asc|desc],

Asc 代表正序, 它是默认选项, 不写就是 asc

Desc 代表倒序

当用一个字段做排序时, 如果出现针对排序字段相同的记录时, 会隐含去使用主键进行第二排序



```
1. SELECT * FROM employee order by salary desc;
2. SELECT * FROM employee order by salary;
3. SELECT * FROM employee order by salary ASC;
```

五、聚合函数

MySQL 有一组函数是特意为求和或者对表中的数据进行集中概括而设计的。这一组函数就叫做聚合函数。聚合函数常常用于对一组值进行计算, 然后返回单个值。通过把聚合函数(如 COUNT 和 SUM) 添加到带有一个 GROUP BY 子句的 SELECT 语句块中, 数据就可以聚合。聚合意味着是求一个和、平均、频次及子和, 而不是单个的值。

1. GROUP BY 子句

主要用于分组。通常在字段表达式中使用分组函数。

格式: group by 字段 1 [asc|desc], 字段 2 [asc|desc],

排序不写默认是 asc 正序

使用 count(*) 和 count(字段名)时, 注意字段的值为 null 的问题

count(字段名)统计时, 会把值为 null 的记录忽略掉!

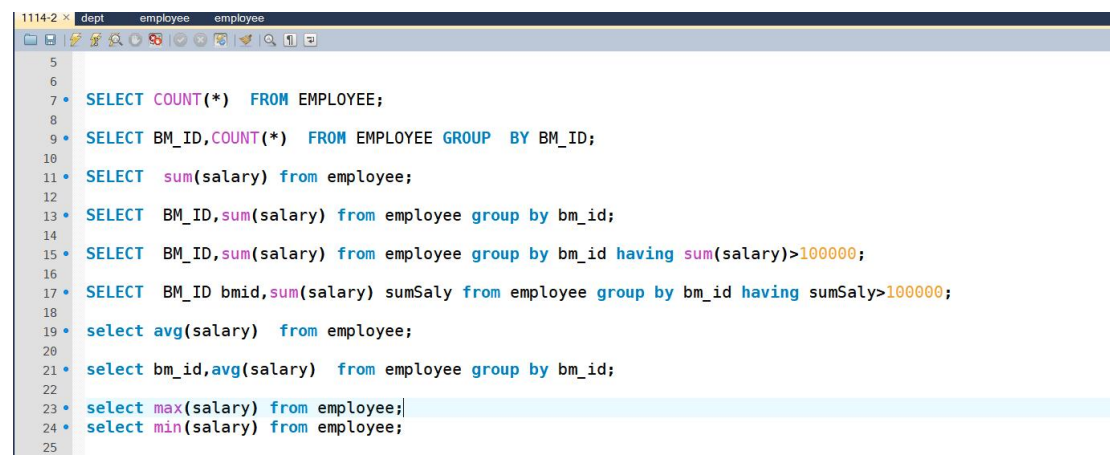
2. HAVING 子句

它用于对查询出来的结果进行再次的过滤。

比较显著的就是如对分组查询出来的结果再次使用条件。

省略 where 和 group by 子句时, 不等于 having 就是 where

不能把 where 放到 having 的位置



```
5
6
7 • SELECT COUNT(*) FROM EMPLOYEE;
8
9 • SELECT BM_ID,COUNT(*) FROM EMPLOYEE GROUP BY BM_ID;
10
11 • SELECT sum(salary) from employee;
12
13 • SELECT BM_ID,sum(salary) from employee group by bm_id;
14
15 • SELECT BM_ID,sum(salary) from employee group by bm_id having sum(salary)>100000;
16
17 • SELECT BM_ID bmid,sum(salary) sumSaly from employee group by bm_id having sumSaly>100000;
18
19 • select avg(salary) from employee;
20
21 • select bm_id,avg(salary) from employee group by bm_id;
22
23 • select max(salary) from employee;
24 • select min(salary) from employee;
25
```

通过 SQL 语句的查询 (select) 可以从一个或多个表或视图中迅速方便地检索数据

SELECT 语句基本格式:

select [选项子句] 字段表达式子句 [from 子句] [where 子句] [group by 子句] [having 子句]
[order by 子句] [limit 子句];

提示: 子句的顺序是固定的

LIMIT 子句

这个子句是做分页的。

格式: limit 起始记录位置, 记录数

注意: 如果省略起始记录位置, 只给出记录数, 表示从第一条数据开始取内容

Limit 10 相当于 limit 0, 10

提示: 记录的位置是从 0 开始计算的

六、Where 子句

带 BETWEEN 、 IN 的范围查询

用于范围比较的关键字有两个: BETWEEN 和 IN。

当要查询的条件是某个值的范围时, 可以使用 BETWEEN 关键字。BETWEEN 关键字指出查询范围, 格式为:

expression [NOT] BETWEEN expression1 AND expression2

当不使用 NOT 时, 若表达式 expression 的值在表达式 expression1 与 expression2 之间 (包括这两个值), 则返回 TRUE, 否则返回 FALSE; 使用 NOT 时, 返回值刚好相反。

```

28 • select * from employee where salary>10000;
29
30 • select id 序号,name 姓名,salary 薪资 from employee where salary>10000;
31
32 • select id 序号,name 姓名,salary 薪资 from employee where salary>=10000;
33
34 • select id 序号,name 姓名,salary 薪资 from employee where salary!=10000;
35
36 • select id 序号,name 姓名,salary 薪资 from employee where salary<>10000;
37
38 • select id 序号,name 姓名,salary 薪资 from employee where salary>=12000 && salary<19000;
39
40 • select id 序号,name 姓名,salary 薪资 from employee where salary>=12000 and salary<19000;
41
42 • select id 序号,name 姓名,salary 薪资 from employee where not (salary>=12000 and salary<19000);
43
44 • select id 序号,name 姓名,salary 薪资 from employee where salary between 12000 and 19000 ;
45
46 • select id 序号,name 姓名,salary 薪资 from employee where salary not between 12000 and 19000 ;
47

```

七、模糊查询

LIKE 运算符用于指出一个字符串是否与指定的字符串相匹配，其运算对象可以是 char、varchar、text、datetime 等类型的数据，返回逻辑值 TRUE 或 FALSE。LIKE 谓词表达式的格式为：

match_expression [NOT] LIKE match_expression [ESCAPE 'escape_character']

使用 LIKE 进行模式匹配时，常使用特殊符号_和%，可进行模糊查询。“%”代表 0 个或多个字符，“_”代表单个字符。

```

51 • select id 序号,name 姓名,salary 薪资 from employee where name='张无忌';
52 • select id 序号,name 姓名,salary 薪资 from employee where name like '张%';
53
54 • select id 序号,name 姓名,salary 薪资 from employee where name like '张_';
55
56 • select id 序号,name 姓名,salary 薪资 from employee where name like '_五';
57
58 • select id 序号,name 姓名,salary 薪资 from employee where name like '_五%';

```

八、子查询

IN 的含义是匹配子查询结果中的任一个值即可（"IN"操作符，能够测试某个值是否在一个列表中）

ANY 要符合子查询结果的任何一个值即可。而且须注意 ALL 和 ANY 操作符不能单独使用，而只能与单行比较符（=、>、<、>=、<=、<>）结合使用

ALL 必须要符合子查询的所有值才可

```

62 • select id,name,salary from employee where salary in (select avg(salary)-1000 from employee where bm_id>=20 group by bm_id);
63
64 • select id,name,salary from employee where salary > any (select avg(salary)-1000 from employee where bm_id>=20 group by bm_id);
65
66 • select id,name,salary from employee where salary > all (select avg(salary)-1000 from employee where bm_id>=20 group by bm_id);
67

```

九、相关子查询 exists

子查询的查询条件依赖于外层父查询的某个属性值的称为相关子查询;将主查询的数据，放到子查询中做条件验证，根据验证结果（TRUE 或 FALSE）来决定主查询的数据结果是否得以保留。

EXISTS 表示存在量词：带有 EXISTS 的子查询不返回任何记录的数据，只返回逻辑值“True”或“False”

```

68
69 • select * from employee where exists (select * from employee where name like '赵%');
70

```

十、多表联合查询

编写 sql 语句时，可以把一个查询结果当表再次查询

```

73 • select count(*) from
74 (select * from employee where exists (select * from employee where name like '赵%')) as emp_info;

```

```

77 • select * from employee;
78 • select * from dept;
79 /* 笛卡尔积 */
80 • select * from employee ,dept;
81
82
83
84
85

```

id	name	id number	rzsi	gender	salary	bm id	dp id	dp name
1001	张无忌	89089123556454	2023-11-14 12:32:01	男	12000.00	10	40	外联部
1001	张无忌	89089123556454	2023-11-14 12:32:01	男	12000.00	10	10	生产部
1001	张无忌	89089123556454	2023-11-14 12:32:01	男	12000.00	10	20	研发部
1001	张无忌	89089123556454	2023-11-14 12:32:01	男	12000.00	10	30	销售部
1003	张三	89125467656454	2023-11-14 12:33:15	男	12000.00	20	40	外联部

```

1114-2 x employee dept
76
77 • select * from employee;
78 • select * from dept;
79 |
80 /* 笛卡尔积 */
81 • select * from employee ,dept;
82 /* 交叉连接 */
83 • select * from employee cross join dept on employee.bm_id= dept.dp_id;
84
85 /* 内连接：按条件完全匹配 */
86 • select * from employee ,dept where employee.bm_id= dept.dp_id;
87
88 /* 内连接：按条件完全匹配 */
89 • select * from employee join dept on employee.bm_id= dept.dp_id;
90
91 /* 左外连接 */
92 • select * from employee left join dept on employee.bm_id= dept.dp_id;
93 /* 右外连接 */
94 • select * from employee right join dept on employee.bm_id= dept.dp_id;
95

```

(1) 内连接 (inner join)

组合两个表中的记录，返回关联字段相符的记录，也就是返回两个表的交集部分

(2) 外连接 (outer join)

● 左(外)连接 (left [outer] join)

以左边表为主：表 1 left join 表 2 on 条件 [where 条件]

左表的记录将会全部表示出来，而右表只会显示符合搜索条件的记录。右表记录不足的地方均为 NULL。

● 右(外)连接 right [outer] join

以右边表为主：表 1 right join 表 2 on 条件 [where 条件]

左表只会显示符合搜索条件的记录，而右表的记录将会全部表示出来。左表记录不足的地方均为 NULL。

(3) LEFT 和 RIGHT 函数

语法格式：LEFT | RIGHT (str ,x)

分别返回从字符串 str 左边和右边开始指定 x 个字符。

(4) TRIM、LTRIM 和 RTRIM 函数

语法格式：

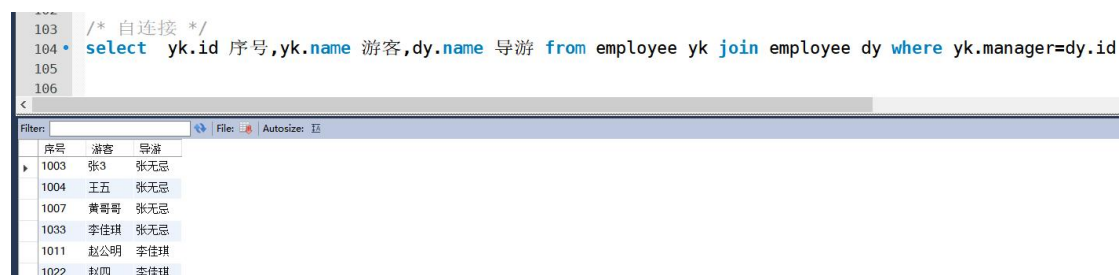
TRIM | LTRIM | RTRIM(str)

使用 LTRIM 和 RTRIM 分别删除字符串中前面的空格和尾部的空格，返回值为字符串。参数 str 为字符型表达式，返回值类型为 varchar。

TRIM 删除字符串首部和尾部的所有空格。

十一、自连接

自连接就是两张表结构和数据内容完全一样的表，在做数据处理的时候，我们通常会给它们分别重命名(别名)来加以区分（言外之意：不重命名也不行啊，不然数据库也不认识它们谁是谁），然后进行关联。



十二、合并

表合并

可以将两个查询语句的结果进行合并拼接，合并拼接的前提是：两个表的表结构一样，

union: 合并拼接时，可以自动去重，剔除重复项（重复项：该项每个字段值完全一样）

union all: 合并拼接时，不能够自动去重，不剔除重复项（重复项：该项每个字段值完全一样）

```
110 select id,name from employee
111 union
112 select dp_id,dp_name from dept;
```

```
107 • create table emp_dp
108 as
109 select id,name from employee
110 union
111 select dp_id,dp_name from dept;
```